CivicSetu — Codebase Architecture & Technical Guide

Comprehensive Guide for Project Review 1 (Architecture, Code Structure, Stack Rationale & Viva Q&A;)

Project: CivicSetu AI Platform	**Generated Date:** August 14, 2026
Target Milestone: Full Core Architecture (10 Modules)	**Test Verification:** 175 / 175 Unit Tests Passing (100%)

1. System Architecture & Design Principles

CivicSetu is designed following the **Clean Service-Layer Architecture** pattern within Django. Unlike standard Django applications that mix database operations directly inside views or serializers, CivicSetu strictly decouples the HTTP layer from business logic using dedicated service modules.

- **Service-Layer Business Logic (services.py):** HTTP Views handle request parsing, serializers perform format validation, while *services.py* executes all business workflows (complaint status transitions, tracking code generation, notification dispatch).
- **Transactional Safety (@transaction.atomic):** Multi-table database operations (e.g. status update + history timeline log + notification creation) are wrapped in atomic transactions so that any runtime failure triggers an immediate automatic rollback.
- **Role-Based Access Control (RBAC):** Fine-grained permission classes (IsCitizen, IsAuthority, IsSuperAdmin) govern every REST API endpoint.
- **Stateless Authentication (JWT):** Uses SimpleJWT tokens containing user identity and role claims, eliminating server-side session overhead.

2. Technology Stack & Architectural Justifications

Technology Layer	Chosen Stack	Architectural Justification (Why We Used It)
Backend Framework	Python 3.12 + Django 5 + DRF	Provides built-in ORM, protection against SQL injection & CSRF, fast REST API serialization, and scalability.
Database Engine	MySQL 8.0 (InnoDB)	Full ACID transactional compliance, foreign key constraints, utf8mb4 encoding, superior to file-based SQLite.
Authentication	SimpleJWT (JSON Web Tokens)	Stateless token authentication allowing independent scaling and seamless integration with web & mobile apps.
API Documentation	drf-spectacular (OpenAPI 3.0)	Auto-generates interactive Swagger UI at /api/docs/ for real-time endpoint testing during reviews.
Frontend Architecture	Vanilla JS + CSS3 SPA	Lightweight Dark Glassmorphism interface with zero node build dependencies (node_modules), easy to deploy.

3. Implemented Modules & Code Structure Deep-Dive

- **config/ (Modular Configuration):** Settings are split into base.py, dev.py, prod.py, and test.py to separate development credentials from production and test environments.
- **apps/accounts/ & apps/users/:** Implements Custom User Model with email login (USERNAME_FIELD = 'email'), role enums (Citizen, Authority, Admin), JWT login/refresh/logout, profile image upload, and password reset.
- **apps/departments/:** Manages 5 municipal sectors (Roads, Water, Electricity, Waste, Drainage), department authority assignments with designations, and department-wise complaint statistics.
- **apps/complaints/:** Handles grievance filing, auto-generates tracking codes (CMP-YYYYMMDD-XXXX), enforces image file size validation (max 5MB, JPEG/PNG/WebP), and manages state transitions (PENDING → IN_PROGRESS → RESOLVED / REJECTED).
- **apps/authority/:** Municipal Authority command dashboard providing real-time metrics, complaint resolution engine with mandatory evidence upload, and notes logging.
- **apps/notifications/:** Dispatches automated in-app notifications whenever a complaint's status changes.

- **apps/feedback/:** Citizen resolution satisfaction ratings (1-5 stars), resolution quality & timeliness metrics, and department/authority feedback aggregation.
- **apps/ai_predictions/:** Automated NLP triage, hazard detection (live wire, fire, flood), urgency scoring (1-100), geospatial Haversine duplicate detection, and city insights.
- **apps/audit_logs/:** Enterprise security audit trail recording user logins, status changes, and administrative actions with IP and User-Agent tracking.
- **frontend/ (SPA):** Responsive Dark Glassmorphism SPA with live search, complaint dropzone modal, interactive timeline modal, and notification drawer.

4. End-to-End Complaint Lifecycle Flow

1. **Authentication:** Citizen logs into SPA ➔ Receives Access & Refresh JWT tokens stored in localStorage.
2. **Filing Complaint:** Citizen submits grievance with title, category, address, and evidence photo.
3. **Service Processing:** ComplaintService.create_complaint() auto-generates tracking code (e.g. CMP-20260727-8A3F), stores image, and logs initial timeline status (PENDING).
4. **Department Routing:** Complaint is automatically assigned to the designated municipal department.
5. **Authority Action:** Department Authority reviews grievance, updates status to RESOLVED, uploads resolution proof image, and inputs resolution notes.
6. **Atomic Execution (@transaction.atomic):** System updates complaint record, appends history log, and dispatches in-app notification to Citizen.
7. **Real-Time Tracking:** Citizen views update in their notification drawer and tracks public progress code.

5. Project Reviewer Viva Q&A; Cheat Sheet

Q1: Why did you build a Custom User Model instead of Django's default User?

Answer: Django's default user model relies on username and lacks domain-specific fields like roles (Citizen/Authority), phone numbers, and UUIDs. We configured a Custom User Model at the project start to use Email authentication and enable clean role-based access control (RBAC).

Q2: Why did you choose JWT authentication over Session cookies?

Answer: Session authentication requires server memory/DB lookups for every request. JWTs are stateless—user identity and role claims are cryptographically signed inside the token, making the API scalable across web and future mobile applications.

Q3: How do you handle database integrity during complaint status transitions?

Answer: We wrap multi-step operations (status change + history log + notification insertion) inside @transaction.atomic in services.py. If any database insert fails, all operations roll back automatically, ensuring data integrity.

Q4: Why separate views.py and services.py in your Django apps?

Answer: We use the Clean Service-Layer Architecture. views.py handles HTTP endpoints and permissions, while services.py contains all business logic. This separation made our backend testable, resulting in 149 / 149 passing automated unit tests.